


École nationale supérieure des Mines de Saint-Etienne

LaTeXProject
Rapport

# TD 1 - Architecture des Processeurs

***Élèves :***
Samy TALEB
Ylies VEYRAT

***Encadrants :***
PONTIE Simon
GOUSSELOT Théophile

18 décembre 2024

# Table des matières

**1 Introduction et mise en contexte** **3**

**2 Banc de registres : 4 x 8 bits** **4**
2.1 Question 1 . . . . . 4
2.2 Question 2 . . . . . 4
2.3 Question 3 . . . . . 4

**3 Simulation et vérification banc registres 8bits** **5**
3.1 Question 4 . . . . . 5
3.2 Question 5 . . . . . 6
3.3 Question 6 . . . . . 7

**4 Banc de registres 32bits** **7**
4.1 Question 7 . . . . . 7
4.2 Question 8 . . . . . 9
4.3 Question 9 . . . . . 9
4.4 Question 10 . . . . . 9
4.5 Question 11 . . . . . 10
4.6 Question 12 . . . . . 10

# 1 Introduction et mise en contexte

Ce TD débutera par une analyse approfondie de l'architecture interne du banc de registres utilisé dans l'implémentation RISC-V RV32I. Dans un second temps, nous établirons la liaison de ce banc de registres avec l'unité arithmétique et logique. Enfin, nous procéderons à l'exécution d'un test visant à décoder nos premières instructions de type registre-registre. L'ensemble de ces opérations sera réalisé au moyen du logiciel LogiSim.

# 2 Banc de registres : 4 x 8 bits

Pour commencer, nous entreprendrons la construction d'un mini banc de registres afin d'approfondir notre compréhension de sa structure et de son fonctionnement. Cette première étape implique la création d'un banc de registres composé de quatre registres huit bits. Ce banc de registres sera conçu pour adresser simultanément deux registres "sources" et lire leurs contenus respectifs de manière asynchrone. De plus, il permettra l'adressage d'un registre "destination" et la synchronisation de l'écriture d'une donnée 8 bits selon l'horloge qui rythme le système.

## 2.1 Question 1

La largeur des entrées et sorties rd_data_i, rs1_data_o et rs2_data_o est de 8 bits car cela correspond à la largeur en bits des données traitées.

## 2.2 Question 2

On se place sur 2 bits (pour pouvoir écrire de 0 à 3), ce qui nous permet alors de s'aiguiller sur 4 registres différents.

## 2.3 Question 3

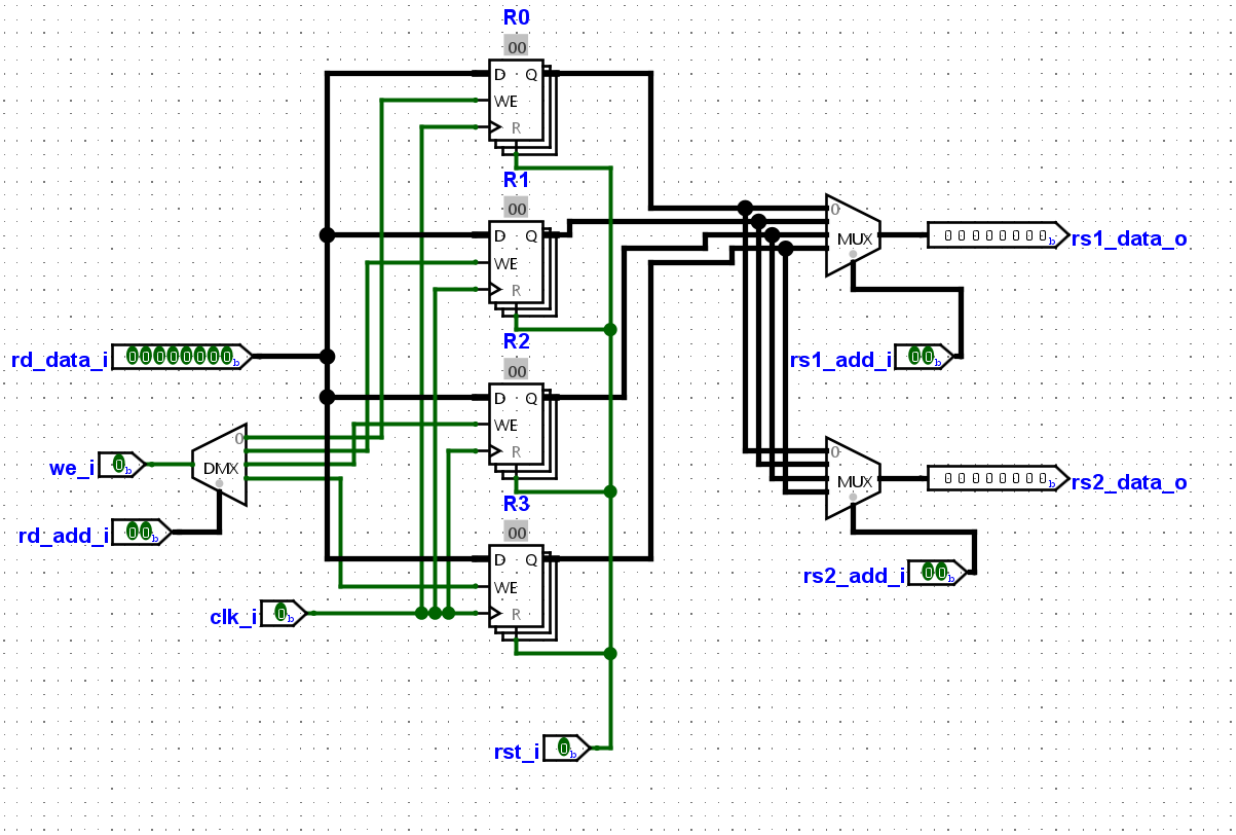



FIGURE 1 – Schéma éléctrique reg8_bank

# 3 Simulation et vérification banc registres 8bits

## 3.1 Question 4

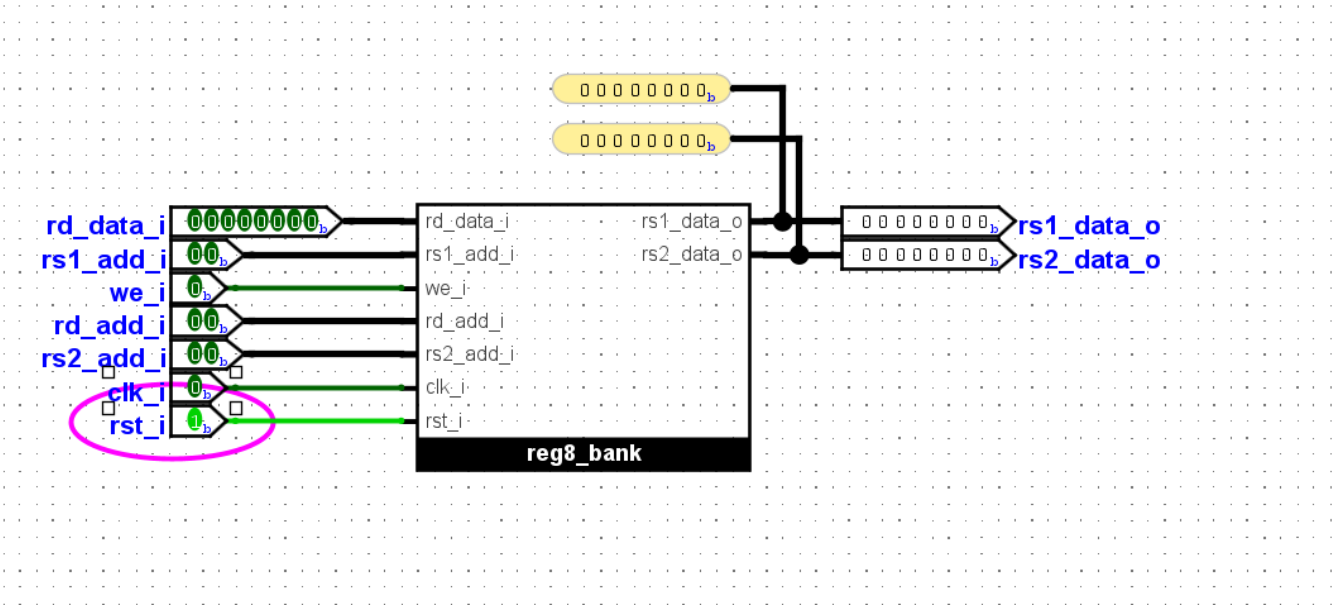



FIGURE 2 – Remise à zéro

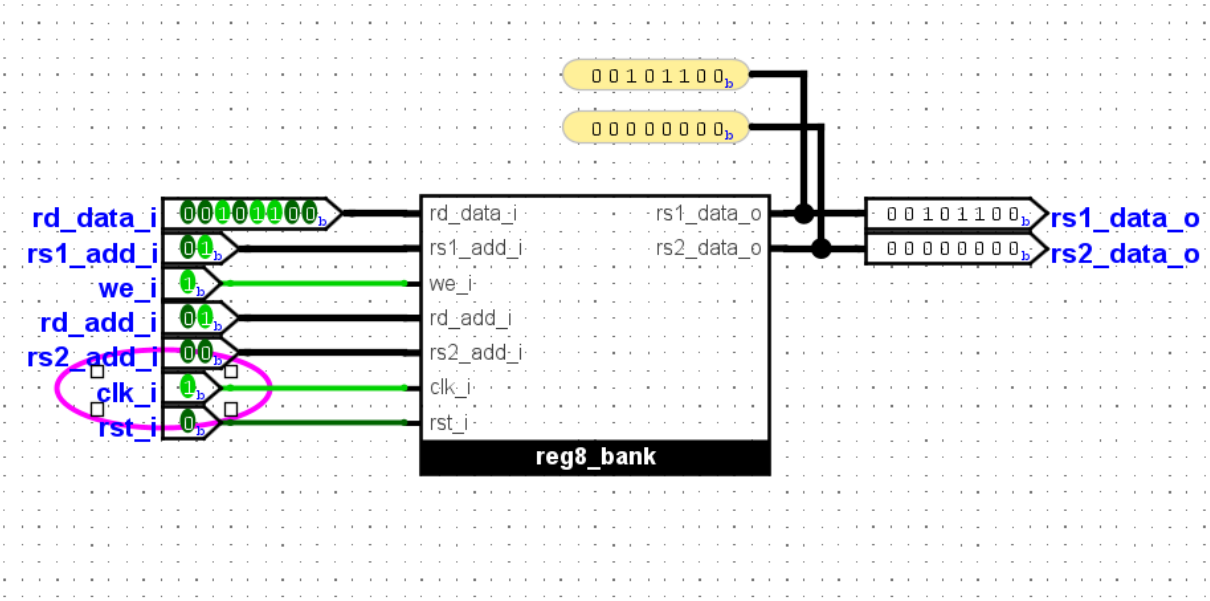



FIGURE 3 – Ecriture synchrone dans plusieurs registres

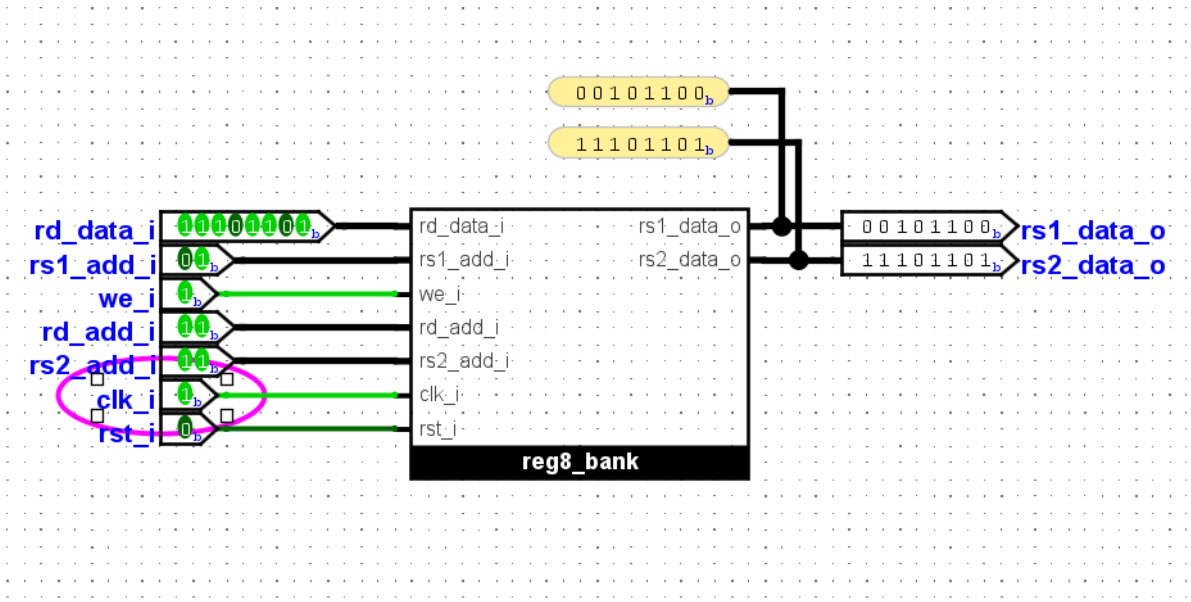



FIGURE 4 – Lecture asynchrone de deux registres précédemment mis à jour lors d'une écriture

## 3.2 Question 5

Voici le chronogramme dessiné :

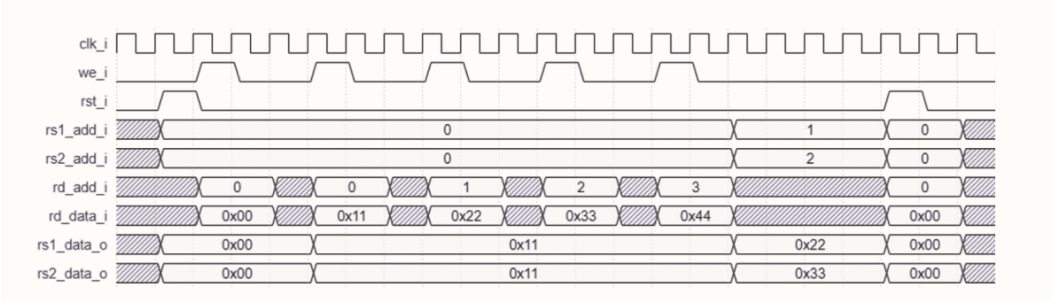



FIGURE 5 – Chronogramme théorique

## 3.3 Question 6

Voici le chronogramme obtenu sur LogiSim.

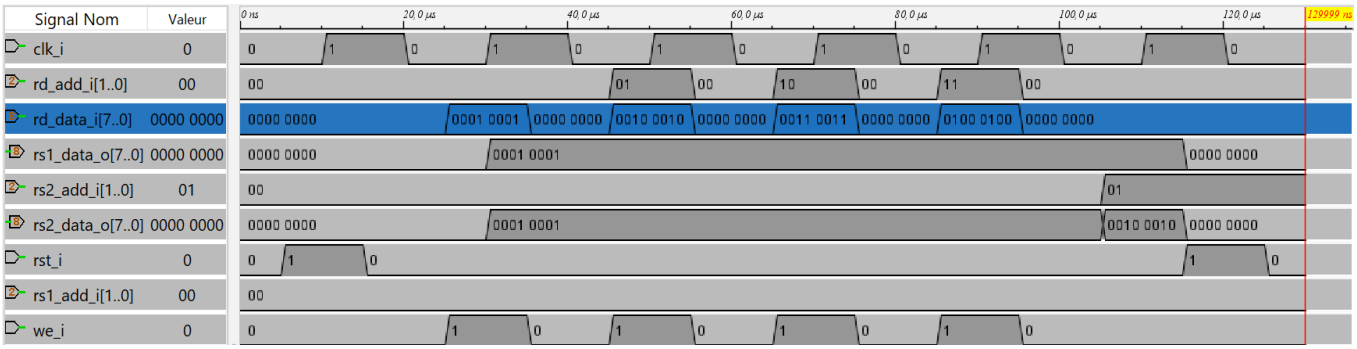



FIGURE 6 – Chronogramme obtenu sur LogiSim

# 4 Banc de registres 32bits

## 4.1 Question 7

On peut s'inspirer des quéstions précédentes pour compléter le schéma registers_bank_incomplé. On obtient ainsi les schémas électriques suivants, où l'on peut observer le branchement des entrées et sorties :

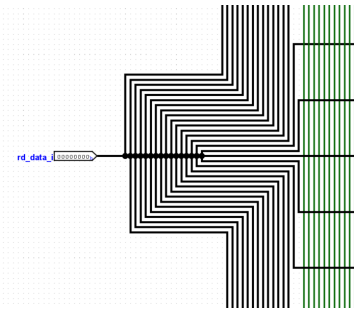


FIGURE 7

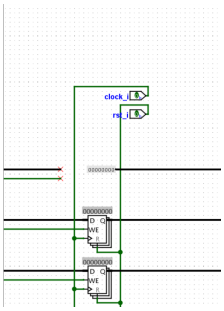


FIGURE 8

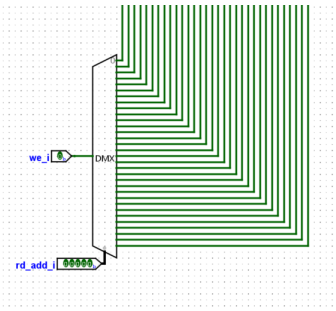



FIGURE 9

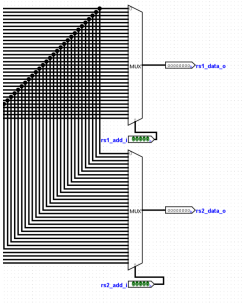


FIGURE 10

On obtient le circuit complet suivant :

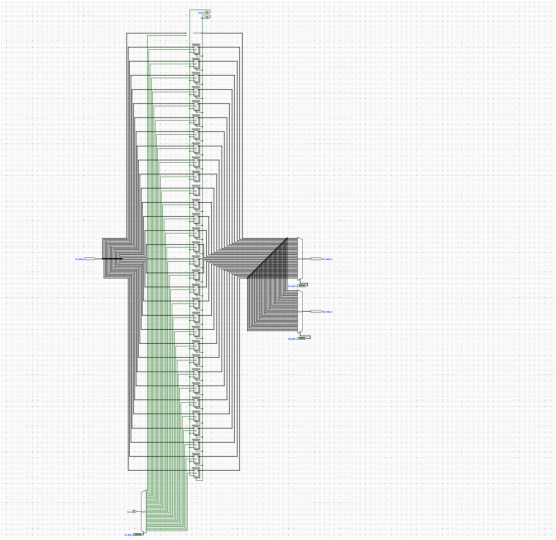


FIGURE 11 – Circuit complet

## 4.2 Question 8

Pour respécter la spécification RISC-V, il faut utiliser le composant Constante de la librairie câblage et de remplacer le registre nul par la constante nulle en déconnectant le câble qui lui est associé comme ceci :

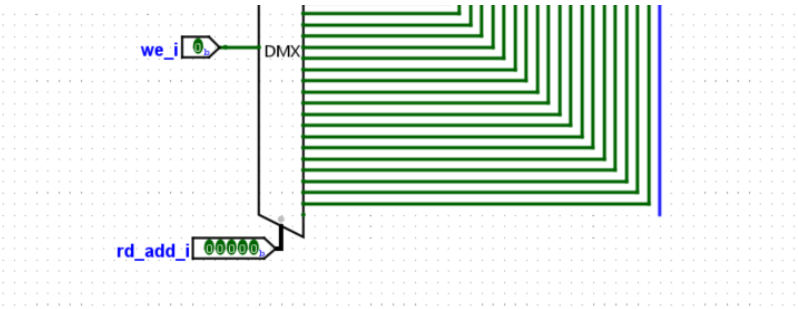



FIGURE 12 – Déconnexion du câble considéré

## 4.3 Question 9

Les entrées de l'ALU et sa sortie sont connectés au banc de registres de la manière suivante :

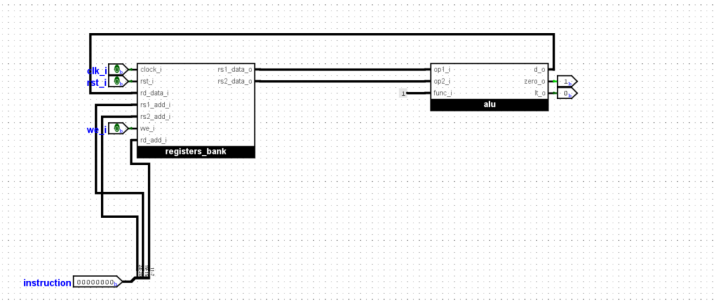



FIGURE 13 – Liaison en série des registres register_bank et alu

## 4.4 Question 10

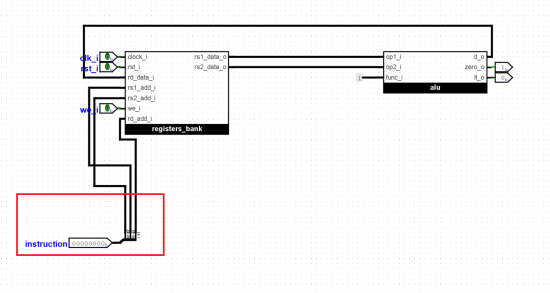



FIGURE 14 – Ajout de l'entrée instruction au schéma électrique

## 4.5 Question 11

Les instructions initiales du code source sont interprétées et converties en langage assembleur à l'aide de commandes spécifiques telles que MOV, qui permettent de charger les valeurs immédiates dans les registres, et ADD, utilisée pour l'addition des valeurs contenues dans les registres. Les données ainsi manipulées sont assignées aux registres R1 à R4, tandis que le résultat final est enregistré dans le registre R5, destiné à stocker la variable e.

| **Instruction C** | **Assembleur** | **Hexadécimal** |
|---|---|---|
| `unsigned int a = 6;` | `MOV R1, 6` | 00608093 |
| `unsigned int b = 2;` | `MOV R2, 2` | 00201013 |
| `unsigned int c = 3;` | `MOV R3, 3` | 00318193 |
| `unsigned int d = 1;` | `MOV R4, 1` | 00120213 |
| `unsigned int e;` | | |
| `e = (a+b) + (c+d);` | `ADD R5, R1, R3` | 002080b3 |
| | `ADD R3, R3, R4` | 004181b3 |
| | `ADD R5, R1, R3` | 003082b3 |

FIGURE 15 – Conversion du code en instructions assembleur, puis passage au format hexadécimal.

## 4.6 Question 12

En appliquant les instructions que l'on vient d'assembler sur l'entrée instruction, on obtient que :

$$R5 = 6 + 2 + 3 + 1 = C \tag{1}$$

***

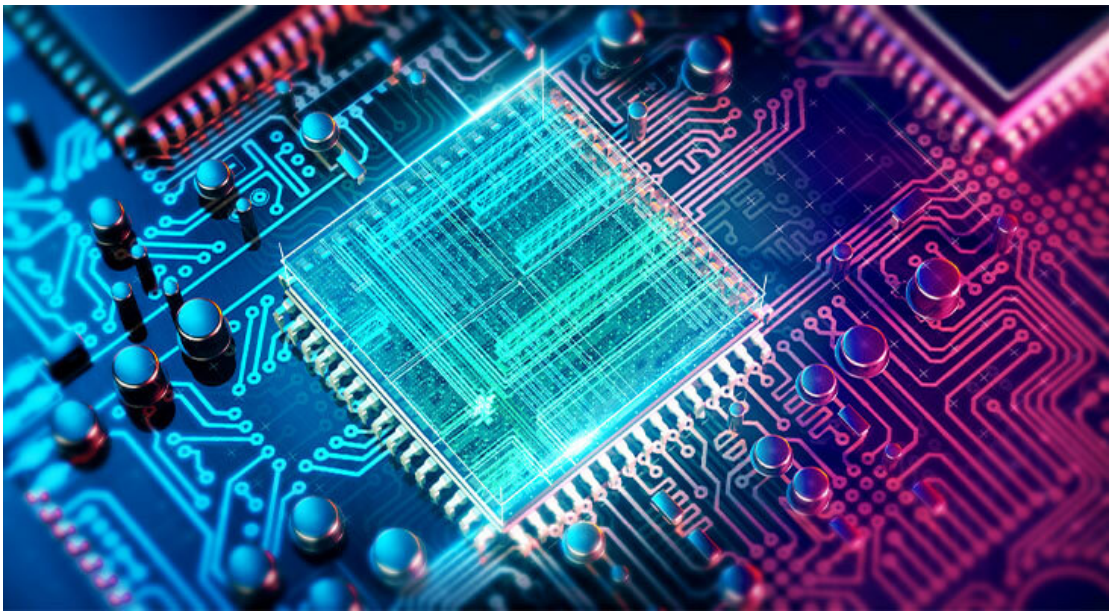


FIGURE 16

# FIN DU RAPPORT

**Merci pour votre attention !**

---

*TALEB Samy, VEYRAT Ylies*